COMP 5521 DISTRIBUTED LEDGER TECHNOLOGY, CRYPTOCURRENCY AND E-PAYMENT
2026 Fall HTLC Project Specification

Objective and Requirements

Build a working **cross-chain asset exchange system** using Hash Time-Locked Contracts (HTLCs) [1]. The system exchanges the custom My Simple Token (MST) on Ethereum Sepolia and native ETH on Arbitrum Sepolia using HTLCs. The two chains do not call each other directly. For the basic project, atomicity must be achieved through a shared hashlock, a revealed preimage and correctly ordered timelocks. The optional Challenge Part may replace this direct linkage with a privacy-preserving construction. A **web-based interface** must guide users through the complete exchange and recovery flow.

The classroom Lab 4 repository [2] (https://github.com/leid19/Lab4-COMP5521-2026) is intentionally a minimal single-chain demonstration. It is a learning reference only and is not a complete submission. Your final project must extend or redesign it into the full two-chain system specified below. Copying the classroom SimpleHTLC as the final architecture does not satisfy the project requirements.

Work in groups of at most **five**. All groups complete the basic requirements in Tasks A–F. The optional Challenge Part is worth the remaining 10 marks. Record each member's contribution and identify incomplete features.

Schedule

Activity	Arrangement
Presentation and demonstration	27 November 2026, 18:30–21:30, FJ303
Report and project submission	Before 4 December 2026, 23:59, via Canvas
Each group	4 minutes slides + 4 minutes live operation + 2 minutes Q&A

Grading

Component	Maximum
A. HTLC smart contracts	25
B. Cross-chain protocol and scripts	20
C. User interface	10
D. Report and presentation	20
E. Live demonstration	10
F. Individual contribution	5
Challenge Part (Optional)	10
Total	100

Tasks A–F cover the complete basic project and are worth 90 marks in total. The separate Challenge Part is worth the remaining 10 marks and is intended for advanced implementations. The Challenge Part is optional. Without it, the maximum available mark is 90/100; the overall project mark remains capped at 100.

Framework

The Lab 4 Hardhat project, the Lab 3 MST deployment and the single-chain HTLC example are provided as learning references. The final project uses `ERC20HTLC.sol` on Ethereum Sepolia for MST and `ETHHTLC.sol` on Arbitrum Sepolia for native ETH. Keep the MST token interface and use public testnet assets; the cross-chain protocol, refund logic, timelock coordination, frontend workflow and supporting scripts are left for implementation.

The two chains do not communicate directly. No bridge, oracle, relayer, cross-chain messaging protocol or production exchange-rate mechanism is required. Shortened timelocks and separate swap instances may be used in the live demonstration so that both successful claim and refund paths can be shown within the presentation time. Grading may include additional accounts, amounts, hashlocks and failure or recovery scenarios under the same contracts.

Task A: HTLC Smart Contract Implementation

25 marks. Implement two self-contained Solidity HTLC contracts that provide the required locking, claiming and refund behavior.

Item	Marks	Required behavior
A1 ERC20 lock	5	Implement MST locking in ERC20HTLC: create a unique swap, transfer MST from the sender after allowance, record sender, receiver, token, amount, hashlock, timelock and status, and emit a creation event. Reject invalid or duplicate swaps.
A2 ETH lock	5	Implement native-ETH locking in ETHHTLC: create a unique swap using msg.value, record the equivalent protocol fields and emit a creation event. Reject zero-value or invalid swaps.
A3 Hashlock claim	6	Implement claim(): allow only the designated receiver to claim an active swap before expiry with a preimage satisfying keccak256(preimage) == hashlock. Transfer the locked asset, prevent repeated completion and emit an event exposing the revealed preimage.
A4 Timelock refund	6	Implement refund(): after expiry, allow only the original sender to recover an unclaimed swap. Reject early refunds, claims after expiry, repeated claims, repeated refunds and refund-after-claim attempts.
A5 Validation and safety	3	Validate addresses, amounts, deadlines and swap identifiers; maintain clear swap states and events/errors; perform external token or ETH transfers safely and protect against reentrancy or equivalent hazards.

Implementation Setting

Task A concerns only the on-chain escrow and state-transition rules. Cross-chain coordination, verification of the counterparty lock, selection and ordering of the two timelocks, secret transfer between chains, frontend workflow and live demonstration logic belong to Task B and Task C. The two HTLC contracts do not communicate directly, and no bridge, oracle, relayer or cross-chain messaging protocol is required.

Task B: Cross-Chain protocol and Scripts

20 marks. Build on the two HTLC contracts from Task A and coordinate the swap actions across Ethereum Sepolia and Arbitrum Sepolia. The coordinator may be implemented with scripts, frontend logic or both, but it must treat the two chains as independent networks.

Item	Marks	Required behavior
B1 Deployment and configuration	4	Deploy ERC20HTLC to Ethereum Sepolia and ETHHTLC to Arbitrum Sepolia. Configure the chain IDs, RPC endpoints, HTLC addresses, MST address and participating accounts without exposing private keys or other secrets.
B2 Successful swap coordination	7	Implement a reproducible swap sequence: generate a secret and hashlock, create the first asset lock, verify its parameters, create the matching lock on the other chain, complete both claims and report the final status. Wait for transaction confirmation and expose the transaction hashes on both networks.
B3 Cross-chain state observation	4	Read the two chains independently and determine the current state of each swap, including whether it is active, claimable, claimed, expired or refundable. Do not assume or simulate direct communication between the two HTLC contracts.

Item	Marks	Required behavior
B4 Timeout recovery	5	Coordinate the refund path when the swap does not complete. Detect expired locks, allow the appropriate sender to recover the asset, and avoid presenting claim or refund actions that are no longer valid for the current on-chain state.

Implementation Setting

Task B implements the cross-chain HTLC protocol across Ethereum Sepolia and Arbitrum Sepolia. Treat the two chains as independent networks: neither HTLC contract may read or invoke the contract on the other chain. Cross-chain coordination is performed off-chain through scripts, frontend logic or both.

Scripts should support repeatable protocol operations such as deployment, secret and hashlock generation, lock creation, parameter verification, state inspection, claim and refund. The frontend may invoke or reproduce the same workflow for the live demonstration. No bridge, oracle, trusted intermediary or cross-chain messaging protocol is required.

Task C: User Interface

10 marks. Build a browser-based interface for operating and observing the cross-chain HTLC workflow. Any modern web framework, or plain HTML/CSS/JavaScript with ethers.js or an equivalent EVM library, may be used. The interface must work with MetaMask or another wallet and must not contain hard-coded private keys. The emphasis is on correct interaction with the HTLC protocol rather than visual complexity.

Item	Marks	Required behavior
C1 Wallet and network connection	2	Connect wallet, display account/network, and support switching between Ethereum Sepolia and Arbitrum Sepolia.
C2 Swap setup and state display	2	Display balances, contract/swap information, amounts, hashlock and timelocks, and support MST approval where required.
C3 Guided swap and recovery workflow	4	Guide the user through locking, verification, counter-lock, claim, completion and refund according to the current on-chain state.
C4 Feedback and usability	2	Show transaction progress, errors, transaction links and clear completed/refunded states.

Implementation Setting

The frontend is the main user-facing interface for the live demonstration. It may interact directly with ERC20HTLC on Ethereum Sepolia and ETHHTLC on Arbitrum Sepolia through the connected wallet, and may reuse utilities or protocol logic developed in Task B.

Task D: Report and Presentation

20 marks. Submit a report of at most six main-text pages; references and the contribution table are excluded. Use four sections: design and implementation, Task A and Task B mechanisms, tests and results, and limitations.

Component	Marks	Assessment
Technical report	10	Implementation and key functions (4); tests and results (4); limitations and discussion (2).
PPT	4	A clear sequence: system design, Task A, Task B, test results and limitations.
Presentation	3	Accurate, understandable explanation with coordinated delivery within four minutes.
Q&A	3	Explain the submitted code, protocol mechanisms and observed results; every member must be available.

Task E: Live Demonstration

10 marks in four minutes. Operate the deployed testnet system in real time. Demonstrate one successful cross-chain atomic swap, including the two asset locks, verification of matching swap parameters, preimage-based claims on both chains and the final swap status (6). Then demonstrate one failure or timeout-recovery case, including an invalid or unavailable action before expiry and a successful refund after expiry (4). Prepare contracts, accounts, testnet funds and configuration before your presentation slot.

Shortened timelocks and separate swap instances may be used so that both the successful and recovery paths can be shown within the available time. The demonstration must use the deployed MST on Ethereum Sepolia and native ETH on Arbitrum Sepolia. Groups should prepare deployed contracts and funded accounts before the demonstration slot. Deployment is not included in the four-minute live demonstration.

Replacing the live demonstration with a prerecorded video receives 0/10 for this component. Screenshots and recordings do not earn live-operation marks.

Optional features (Challenge Part) are assessed within Tasks A and B. They are not required for full live-demonstration marks. Keep the two-minute Q&A separate from the four-minute live demonstration.

Task F: Individual Contribution

5 marks per student: a clear description of personal modules, functions, tests or analysis (2), supported by verifiable code, commits or other work products (3). Include one group contribution table with names, student IDs, responsibilities and evidence. Q&A marks assess answers; contribution marks assess the work performed.

Challenge Part: Privacy-Preserving Cross-Chain HTLC

10 marks (OPTIONAL). Extend the basic HTLC protocol to reduce the direct on-chain linkability between the Ethereum Sepolia and Arbitrum Sepolia swap transactions. A standard HTLC exposes the same hashlock and related preimage across both chains; the challenge implementation must avoid publishing the same hashlock or claim preimage on both networks while preserving the atomic claim and timeout-refund properties of the basic protocol.

Item	Marks	Required behavior
CH1 Private linked hashlocks	4	Avoid reusing the same public unlocking value on both chains. Provide a zero-knowledge proof or equivalent cryptographic evidence that the two locks are correctly related without revealing the unrevealed secret. The counterparty must verify this evidence before creating the second lock.
CH2 Atomic private claim flow	4	Complete a successful cross-chain swap in which the first claim reveals enough information for the intended counterparty, together with its private protocol data, to derive or authorize the second claim, while an ordinary public observer cannot directly do so. Preserve receiver authorization, timelock ordering and timeout refunds. An invalid proof, incorrect private value or unrelated hashlock pair must not lead the counterparty to commit the second asset.
CH3 Privacy and security evaluation	2	Demonstrate and explain what information is visible on both chains, why the two claims no longer expose the same hashlock/preimage, and which metadata may still allow correlation. Include at least one failed verification or incorrect-secret case.

Implementation Setting

The Challenge Part concerns cross-chain linkage privacy, not complete transaction anonymity. Account addresses, transaction amounts, contract addresses, block times and other public blockchain metadata may remain visible.

The privacy mechanism must not introduce a trusted bridge, oracle or intermediary. Off-chain exchange of private protocol data between the two swap participants is allowed, but one party must not be required to trust the other to construct correctly related locks.

The zero-knowledge system and cryptographic libraries are not prescribed. Circom/snarkjs, Noir or another suitable framework may be used. Alternative approaches, including adaptor-signature or point-lock constructions, may

receive full marks if they provide equivalent atomicity and cross-chain linkage protection and are clearly documented.

The Challenge Part must build on a working basic implementation. A privacy extension that removes the required timeout recovery, allows one party to claim both assets unfairly, or relies only on hiding values in the frontend does not satisfy the challenge.

Submission

Upload one COMP5521_HTLC_GroupXX.zip archive to Canvas containing:

- **contracts/** containing the final ERC20HTLC.sol and ETHHTLC.sol contracts, together with any shared base contract or supporting Solidity files.
- **scripts/** containing the deployment and protocol scripts used by the project, such as secret/hashlock generation, contract deployment, lock creation, swap inspection, claim and refund operations on both testnets.
- **.env.example** or an equivalent configuration template showing the required RPC URLs, contract addresses, MST address and account-related settings using placeholder values only.
- **Frontend source files** and dependency/configuration files required to run the web interface, for example src/, public/, package.json and the relevant build configuration.
- **README.md** with installation instructions, network configuration, compile/test commands, deployment steps, frontend launch instructions, deployed contract addresses and a reproducible live-demonstration procedure for both successful swap and refund scenarios.
- **Report.pdf, presentation.pdf or presentation.pptx, and the contribution table.**
- **Changes.md** or equivalent section in the README identifying the main differences from the Lab 4 teaching example, completed optional extensions and any known incomplete requirements.

Do not submit virtual environments, dependency folders or a demonstration video. Explain significant changes and include appropriate tests or evidence for any claimed Challenge Part implementation.

Warning and References

AI Use Policy

AI may be used only to **assist the implementation or debugging of individual functions.** Students must **not** use AI agents to implement the entire project or a complete project component.

All AI-assisted code **must be clearly marked**, and the report must state **which functions used AI and what assistance was provided**.

Penalties:

- Entire project/component completed by AI: 0 marks for that project/component.
- AI-assisted code not disclosed: deduction of the corresponding marks.
- Student cannot explain an AI-assisted function in the report/assessment: deduction up to all marks for that function.

All reports will undergo the University's AI-detection checks; please ensure that your work complies with PolyU's academic integrity requirements [3].

[1] Poon J, Dryja T. The bitcoin lightning network: Scalable off-chain instant payments[EB/OL].(2016-1-14)

[2] Teaching repository and baseline README: https://github.com/leid19/Lab4-COMP5521-2026

[3] PolyU academic integrity guidance: https://www.polyu.edu.hk/ogur/docdrive/Academic_Integrity/Plagiarism_Booklet.pdf